

Linux Commands:

1. Answer the following questions in one or two sentences each:

- [½ point] What does the man pwd command tell you about the pwd command? You should learn what the man command is if you tried with different options, such as man ls, man cd.
 - The man pwd provides a manual about the pwd command, which writes the pathname of the current working directory to the standard output. In other words, pwd shows the current working directory users are currently working in. There are two primary options for using the pwd command:
 - -L, which displays the logical current working directory
 - -P, which displays the physical current working directory
- [1½ point] Explain piping in Linux.
 - Piping in Linux is a technique to redirect one command to another for further processing. You can do this by using the pipe character “|” to combine two or more commands. The output of one command is passed to the second command and can continue through multiple commands. Overall, piping allows for processing complex data streams more efficiently, specifically ones that require more specific outputs.
- [3 points] Explain the concepts of stdin, stdout, and stderr.
 - Stdin stands for standard input and it is used for receiving input data. It allows text data input. The number 0 is used to represent Stdin data streams.
 - Stdout stands for standard output and it is used for displaying output data from commands. The result text output is stored in the stdout stream. The number 1 is used to represent stdout data streams.
 - Stderr stands for standard error, which means outputting error messages when a command encounters an issue. It is handled separately from regular output. The number 2 represents stderr data streams.

Handling Compressed Data:

2. [7½ points] Find the column names in the Opioid dataset. The naive way is to gunzip the .gz file and run head -1 on the result, but you likely don't have enough disk space. Conveniently, zcat can read the file and write the unzipped contents into stdout, which can be piped into head -1.

- The command I used was:
zcat arcos_all_washpost.tsv.gz | head -1
 - **zcat** - used to decompress the .gz files and output the data to standard output (stdout)
 - **arcos_all_washpost.tsv.gz** - the compressed file I am working with
 - **|** - Pipe, which takes the output from the command on its left (zcat arcos_all_washpost.tsv.gz) and sends it as input to the command on its right (head)
 - **head** - used to view the beginning portion of the file
 - **-1** - used to display the first first line of the file, which is the names of all the columns of the dataset
- I also output the first line of the header by using this command: **zcat arcos_all_washpost.tsv.gz | head -1 > header.txt**

Here is a list of all the column names in the Opioid dataset:

- REPORTER_DEA_NO
- REPORTER_BUS_ACT
- REPORTER_NAME
- REPORTER_ADDL_CO_INFO
- REPORTER_ADDRESS1
- REPORTER_ADDRESS2
- REPORTER_CITY
- REPORTER_STATE
- REPORTER_ZIP
- REPORTER_COUNTY
- BUYER_DEA_NO
- BUYER_BUS_ACT
- BUYER_NAME
- BUYER_ADDL_CO_INFO
- BUYER_ADDRESS1
- BUYER_ADDRESS2
- BUYER_CITY
- BUYER_STATE
- BUYER_ZIP
- BUYER_COUNTY
- TRANSACTION_CODE
- DRUG_CODE
- NDC_NO
- DRUG_NAME
- QUANTITY

- UNIT
- ACTION_INDICATOR
- ORDER_FORM_NO
- CORRECTION_NO
- STRENGTH
- TRANSACTION_DATE
- CALC_BASE_WT_IN_GM
- DOSAGE_UNIT
- TRANSACTION_ID
- Product_Name
- Ingredient_Name
- Measure
- MME_Conversion_Factor
- Combined_Labeler_Name
- Revised_Company_Name
- Reporter_family
- dos_str

3. [7½ points] Find the number of rows in the Opioid dataset by processing the zcat output, stripping the header row, and counting the remaining lines using wc.

- The number of rows is 178,598,026
- The command I used was: **zcat arcos_all_washpost.tsv.gz | awk 'NR > 1' | wc -l**
 - **zcat** - used to decompress the .gz files and output the data to standard output (stdout)
 - **arcos_all_washpost.tsv.gz** - the file I am working on
 - **|** - Pip, which takes the output from the command on its left (zcat) and sends it as input to the command on its right (head)
 - **'NR > 1'** - awk specifies lines where NR is greater than 1, where the row header is skipped
 - **wc -l** - counts the number of lines that are output from the previous command, awk 'NR > 1'
- **zcat arcos_all_washpost.tsv.gz | wc -l** also worked after using Tufts EECS
 - This resulted in 178,598,027, which can be subtracted by one to get 178,598,026 rows that exclude the header
 - **wc** - stands for “word count”
 - **-l** - tells **wc** to count all the lines

4. [10 points] Find the names of all the drugs named in the dataset.

- The names of all the drugs are:
 - HYDROCODONE
 - OXYCODONE

- I first used a command that would extract column 24, where DRUG_NAME is stored. This way it would be easier to process by focussing on a single column. I used the command: **zcat arcos_all_washpost.tsv.gz | cut -f24 > column__24.txt**
 - **cut** - to extract a specific portion of the file
 - **-f24** - specifies cut to extract the 24th column from the file
 - I want to cut this column because that is where DRUG_NAME is stored
 - **> column__24.txt** - creates a output file where the 24th column data will be saved
- Then I used this command: **sort -u column__24.txt > unique__column__24.txt** to remove all the duplicates
 - **sort** - sort through the file
 - **-u** - specifies sort to look for only unique lines, removing duplicates
 - **> unique__column__24.txt** - save the sorted lines in a different file that does not have any duplicates

5. [10 points] Estimate the number of rows for each year in the dataset. There may be enough space in the shell, but we'd prefer not to rely on that. So here's a potential strategy: Use the shuf command to extract, say, random 7,500 rows from the output of zcat. Find the proportion of rows for each year in this extract. Assuming that the distribution of the random 7,500 rows is similar to the distribution in the whole file, estimate the number of rows for each year.

- I used this command to get a random sample of 7,500 rows:
 - **zcat arcos_all_washpost.tsv.gz | shuf -n 7500 > sample_data.tsv**
 - **shuf -n 7500** - randomly shuffles the lines of the rows in the arcos_all_washpost.tsv.gz files and selects only 7,500 rows
 - **> sample_data.tsv** - output file of where the sample is saved under
- Next, I used this command to extract only the TRANSACTION_DATE, which is in column 31:
 - **cut -f31 sample_data.tsv > transaction_dates.txt**
- I noticed the TRANSACTION_DATE column has the date organized like this, 01082008 (MMDDYYYY), but I only want the year. So, I created an output file to extract only the year from the column:
 - **cut -c5-8 transaction_dates.txt > years.txt**
 - **-c5-8** - is used to extract everything that is in position 5 to 8, which is the year
- Finally, I sort out the years by counting how many times each year appears in the sample:
 - **sort years.txt | uniq -c > year_counts.txt:**
 - **883 - 2006**
 - **994 - 2007**
 - **1013 - 2008**
 - **1032 - 2009**
 - **1164 - 2010**

- 1162 - 2011
- 1252 - 2012

- Here is the portion of rows for each year in the **sample**:

Sample		
Total Rows:	7500	
Count of Years	Year	Proportion to Total
883	2006	11.8%
994	2007	13.3%
1013	2008	13.5%
1032	2009	13.8%
1164	2010	15.5%
1162	2011	15.5%
1252	2012	16.7%
7500		100.0%

- Using the proportion of the years from the sample, I used the same proportion percentage for the whole file that contains 178,598,026 rows, excluding the header:

Whole		
Total Rows:	178598026	
Year	Proportion from Sample	Count of Years
2006	11.8%	21026941
2007	13.3%	23670192
2008	13.5%	24122640
2009	13.8%	24575088
2010	15.5%	27718414
2011	15.5%	27670787
2012	16.7%	29813964
		178598026

6. [10 points] Extract any 4,000 rows for the year 2012 from arcos_all_washpost.tsv.gz. Write the resulting lines to arcos_2012.tsv.

- To extract the year from the TRANSACTION_DATE column in arcos_all_washpost.tsv.gz, which is in MMDDYYYY format, I used a command that extracts only the YYYY portion, which is located in positions 5 to 8 of the date string, similar to the approach I used in question 5.

